Title:

CubeLand (3D Open-World Terrain using OpenGL)

1. Introduction

CubeLand is a 3D open-world terrain simulation built using OpenGL and C++. It is inspired by voxel-based games like Minecraft and was developed as a semester project for the HCI and Computer Graphics course.

The project demonstrates core computer graphics concepts including 3D rendering, camera systems, lighting, raycasting, collision detection, and procedural world generation, all implemented from scratch using low-level OpenGL APIs.

The player can freely explore a randomly generated block-based world, place and remove blocks, build rooms, and interact with the environment in real time.

2. Problem Statement

Modern 3D graphics engines and game engines abstract away the underlying rendering pipeline to the point where students rarely get to engage with the fundamental concepts directly. Tools like Unity or Unreal handle lighting, camera systems, collision, and rendering automatically, leaving little room to understand what actually happens at the GPU level.

This project addresses that gap by building a functional 3D interactive world entirely from scratch using raw OpenGL APIs, with no game engine or high-level framework involved. Every system, the camera, lighting, terrain, collision detection, block

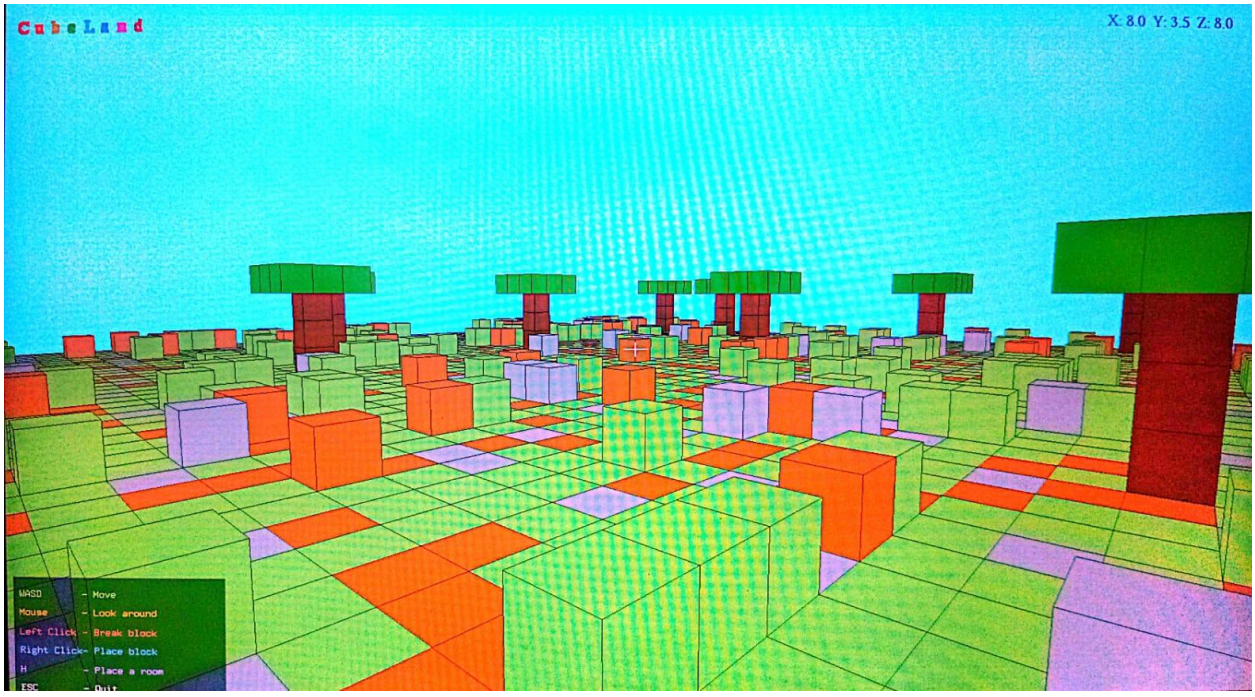
interaction, and HUD, is implemented, forcing a direct understanding of how each concept works at the code level.

The goal is not to replicate Minecraft, but to use the voxel-based format as a familiar and visual framework for applying and demonstrating core computer graphics principles including the rendering pipeline, the Phong lighting model, geometric transformations, raycasting, and procedural generation, in a way that is interactive and immediately visible to the user.

3. Main Interface

The main interface consists of a fullscreen 3D first-person view of the world. The following elements are always visible on screen:

- Top-left: CubeLand title in colorful letters
- Top-right: Live XYZ coordinates of the player
- Bottom-left: Control reference panel with color coded labels
- Center: Crosshair for block targeting



[Screenshot: Main Interface — Front View]

4. Tools Used

- VS Code — code editor with custom tasks.json build configuration
- MSYS2 / MinGW64 — compiler toolchain for Windows
- OpenGL — graphics API for rendering
- C++ — primary programming language

5. Libraries Used

- GLFW3 — window creation, input handling, OpenGL context
- GLU — utility library for gluPerspective and gluLookAt
- GLUT / Freeglut — bitmap font rendering for HUD text
- GLEW — OpenGL extension loading
- cmath — trigonometry for camera direction vectors
- ctime — seeding random number generator for world generation
- cstdio — sprintf for formatting coordinate strings

6. Key Features

- Randomly generated world each run — grass, dirt, and stone blocks
- Height variation — elevated terrain and hills
- First-person WASD movement with mouse look
- Block placing (right click) and breaking (left click)
- DDA raycasting for precise block targeting at crosshair
- Collision detection — wall sliding, height restriction
- Trees placed randomly across the terrain
- Place a pre-defined room structure by pressing H
- Live XYZ coordinates displayed on screen
- Fullscreen at native monitor resolution

7. Extra Tweaks (Hardcoded Config)

The following values can be changed at the top of the code to adjust behavior without touching any logic:

- WORLD_SIZE — size of the terrain grid (e.g. 16, 32, 64)
- WORLD_SCALE — scale multiplier for block size
- CAM_HEIGHT — how high above ground the camera sits (default 1.5)
- PLAYER_SPEED — movement speed (default 5.0)
- sensitivity — mouse look sensitivity (default 0.1)
- MAX_TREES — number of trees placed in the world (default 8)
- placeHome layout[] — character map defining room structure (W=wall, D=door, .=floor)

8. Computer Graphics Concepts Used

- Translation — glTranslatef to position each block in world space
- Matrix stack — glPushMatrix / glPopMatrix for isolated transforms
- Yaw and Pitch — camera rotation angles for first-person look
- View matrix — gluLookAt positions the camera in 3D space
- Perspective projection — gluPerspective with 60 degree FOV
- Orthographic projection — glOrtho for 2D HUD overlay
- Flat shading — glShadeModel(GL_FLAT), one color per face
- Ambient lighting — low-intensity base light (0.3) so no face is fully black
- Diffuse lighting — directional light (1.0) shading faces by angle to light source
- Face normals — glNormal3f per face for lighting calculations
- Depth buffering — GL_DEPTH_TEST ensures correct front-to-back rendering
- Alpha blending — GL_BLEND for semi-transparent HUD panel
- Polygon offset — GL_POLYGON_OFFSET_FILL prevents z-fighting on outlines
- DDA raycasting — precise block face detection for interaction
- Double buffering — glfwSwapBuffers for flicker-free rendering

9. Lighting Equation

The general Phong lighting equation is:

$$I = Ia \cdot ka + Id \cdot kd \cdot (N\hat{} \cdot L\hat{}) + Is \cdot ks \cdot (R\hat{} \cdot V\hat{})n$$

In this project, specular lighting is not used (Is = 0), so the equation simplifies to:

$$I = (0.3) \cdot (\text{block color}) + (1.0) \cdot (\text{block color}) \cdot (\hat{N} \cdot \hat{L})$$

Variable mapping:

- Ia = 0.3 — ambient light intensity (glLightfv GL_AMBIENT)
- Id = 1.0 — diffuse light intensity (glLightfv GL_DIFFUSE)
- Is = 0 — specular not used, term drops out
- ka = kd = block's glColor3f value (GL_AMBIENT_AND_DIFFUSE)
- N = face normal vector (glNormal3f, e.g. top face = 0,1,0)
- L = direction to light (GL_POSITION = 1.0, 2.0, 1.0, 0.0 — directional)
- N.L = dot product — determines how directly a face points at the light

Example — top face of a grass block (color = 0.4, 0.7, 0.3):

- N = (0, 1, 0), L normalized = (0.41, 0.82, 0.41)
- N.L = 0*0.41 + 1*0.82 + 0*0.41 = 0.82
- I = (0.3)(0.4, 0.7, 0.3) + (1.0)(0.4, 0.7, 0.3)(0.82)
- I = (0.12, 0.21, 0.09) + (0.33, 0.57, 0.25) = (0.45, 0.78, 0.34)

10. Basic Process

The entire world is built from a single drawCube function. Everything else calls it:

- drawCube(x, y, z, r, g, b) — draws one 1x1x1 colored cube with black outlines
- drawTerrain() — loops through world[][] and height[][] arrays, calls drawCube per block per layer
- drawTree(x, z) — calls drawCube for 3 trunk blocks and a 3x3 leaf canopy on top
- placeHome(ox, oz) — reads a layout[] character map, calls drawCube to build walls and floor

This means adding a new structure type is always the same pattern: define coordinates, call drawCube.

11. Rendering Pipeline

Every frame the following steps execute in order:

- 1. Clear screen: wipe color buffer and depth buffer from last frame
- 2. Snap camera to terrain: player automatically steps up/down with terrain
- 3. Set up perspective projection: gluPerspective with 60 degree FOV
- 4. Draw 3D world: terrain, trees, placed blocks and rooms
- 5. Switch to 2D mode: glOrtho maps coords to screen pixels
- 6. Draw 2D: crosshair, control panel, CubeLand title, XYZ coordinates
- 7. Swap buffers: glfwSwapBuffers flips back buffer to screen
- 8. Poll events: keyboard, mouse, window close checked for next frame

12. Block Placing and Breaking

Block interaction uses DDA (Digital Differential Analyzer) raycasting. When a mouse button is clicked:

- A ray is cast from the camera position in the look direction
- The ray steps exactly to each block boundary (no floating point drift)
- The first solid block hit is recorded as the target (hitX, hitY, hitZ)
- The last empty cell before the hit is the placement position (placeX, placeY, placeZ)
- Left click — decrements height[hitX][hitZ] by 1 (top block removed)
- Right click — increments height[placeX][placeZ] by 1 (new block added on top)

Known limitation: blocks are stored as height columns (2D heightmap), not individual 3D cells. Breaking always removes the top of a column. A full 3D array (world[x][y][z]) would be needed for mid-column editing.

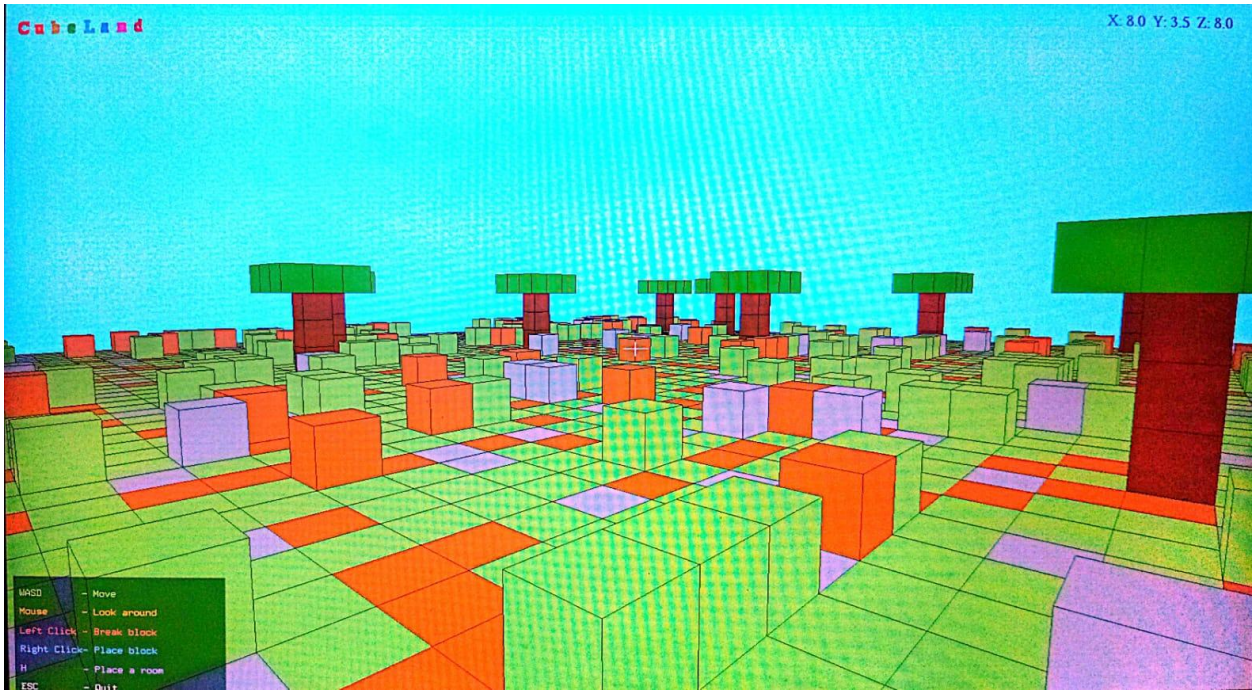
13. Collision Detection

Collision is handled via a pre-baked solid[][] array populated at startup:

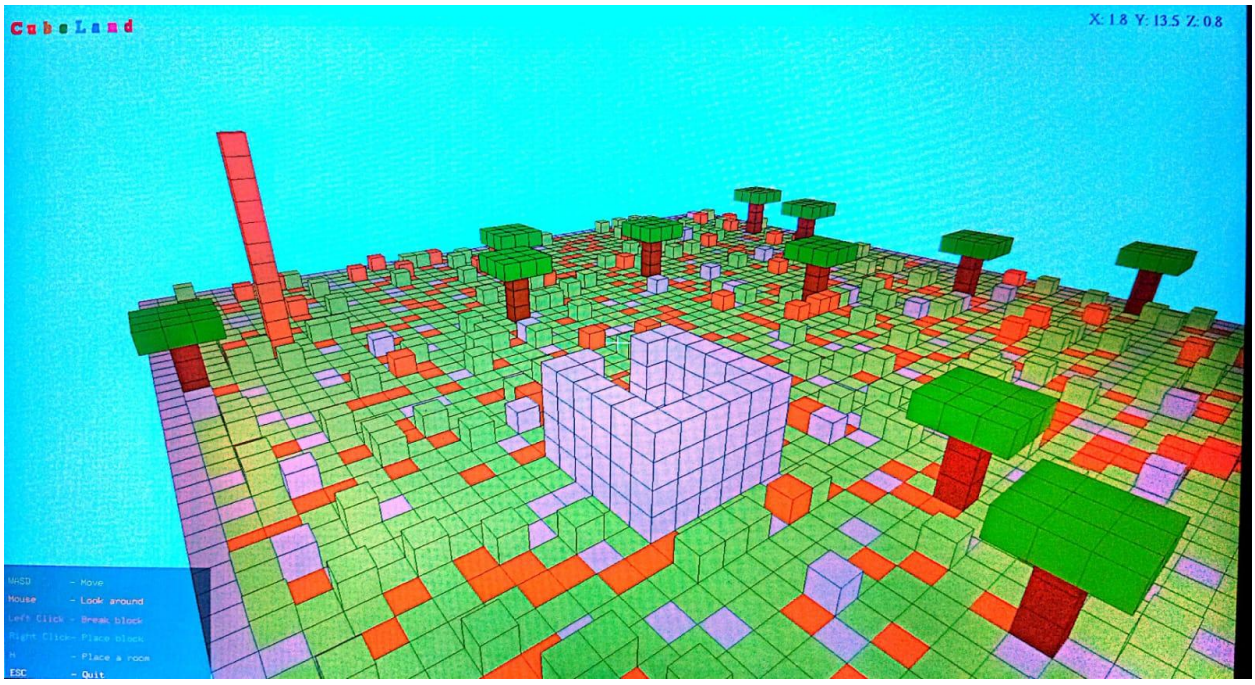
- Elevated terrain tiles (height > 1) are marked solid
- Tree trunk positions are marked solid
- Out-of-bounds positions always return solid
- Player's current tile is cleared every frame so the player never gets trapped

Movement uses axis-separated collision response: if the full move is blocked, X and Z are tried independently so the player slides smoothly along walls instead of stopping dead.

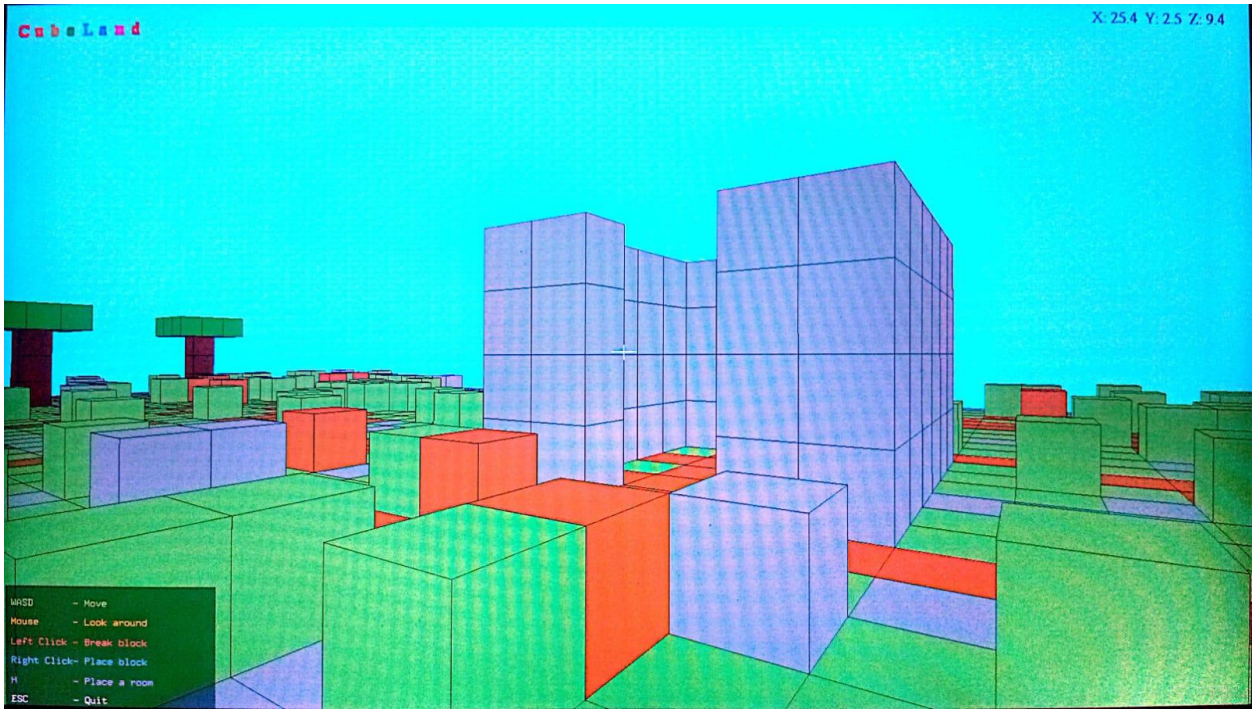
14. Screenshots



[Screenshot: Main View — Walking around terrain]



[Screenshot: Top View — Aerial perspective of world]



[Screenshot: Place Room (H) — Room structure in world]]

15. Conclusion

CubeLand successfully demonstrates a range of core computer graphics and HCI concepts in a functional interactive application. The project covers the full pipeline from raw OpenGL primitives to a playable first-person 3D world.

Key learning outcomes include understanding the OpenGL matrix stack, implementing a first-person camera from scratch, applying the Phong lighting model, and building interactive raycasting-based block manipulation.

Possible future improvements include a full 3D voxel array for per-block editing, texture mapping, dynamic day/night lighting, and larger procedurally generated worlds using noise functions.